Бинарное дерево поиска

Что такое бин. дерево поиска? Чем отличается от обычного бинарного дерева?
Бинарное дерево - структура данных, в которой у каждого узла не более двух потомков
Бинарное дерево поиска - бинарное дерево, у которого все узлы левого поддерева меньше или равны корню, а все узлы правого поддерева больше.

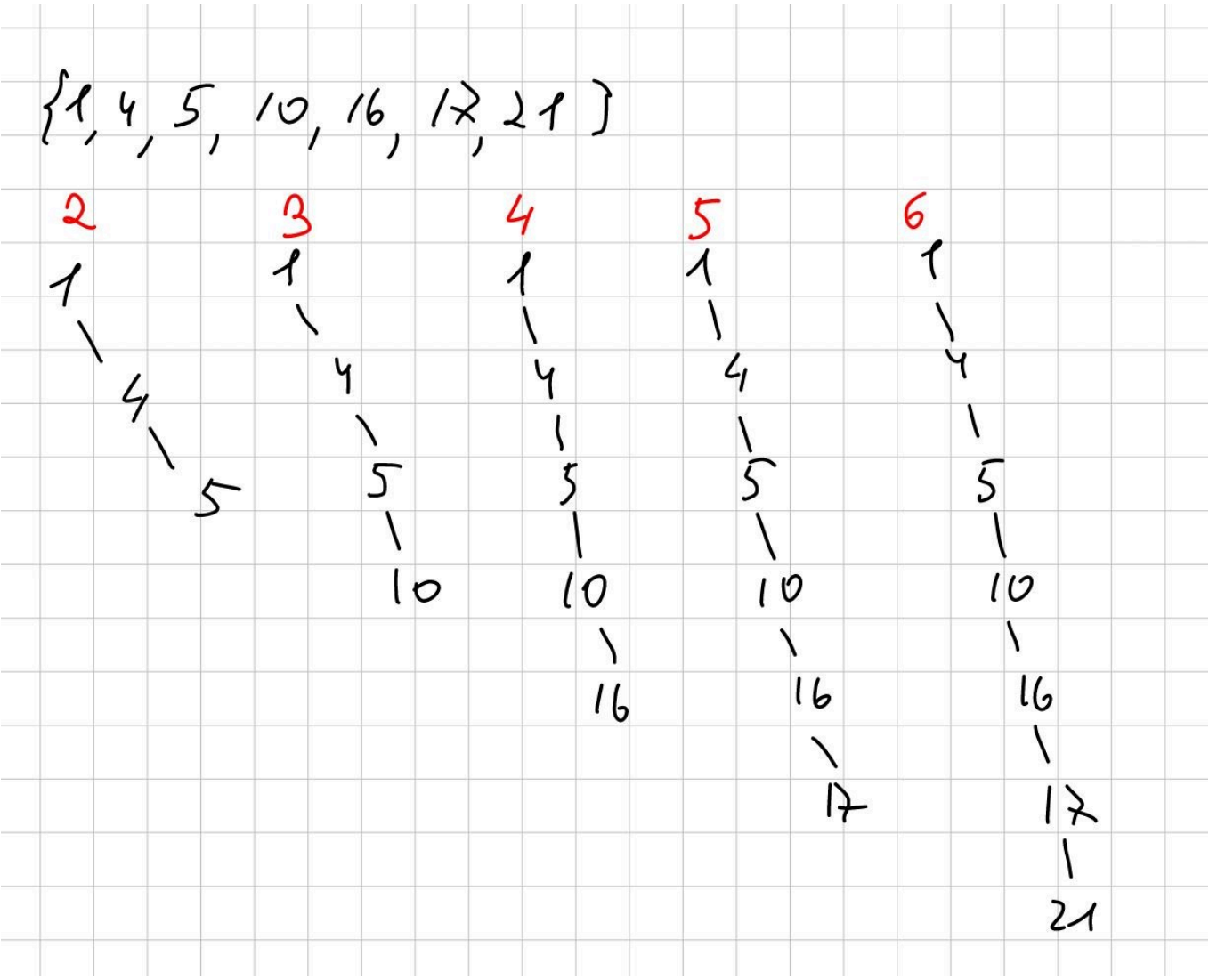
> Простое бинарное дерево определяет структура, бинарное дерево поиска вводит правило на расположение элементов

Операции и их сложность

- Вставка: $O(h) = O(\log n)$ - в лучшем/ среднем, $O(h) = O(n)$ в худшем (когда дерево становится похожим на массив)
- Удаление:
  1. Узел лист - просто удалеям
  2. У узла один ребёнок, ставим ребёнка на место удаляемого узла
  3. У узла два ребёнка, находим замену (максимальное в левом поддереве или минимум в правом), ставим найденный узел вместо этого, исходный удаляем, переставленный удаляем там, откуда забрали его.
- Поиск: $O(h) = O(\log n)$ - в лучшем/ среднем, $O(h) = O(n)$ в худшем
- Поиск минимум/максимума, предыдущего/следующего элемента: $O(h) = O(\log n)$ - в лучшем/ среднем, $O(h) = O(n)$ в худшем
- Повороты: $O(1)$, т.к. работаем напрямую с указателями.
- Обходы (preorder, inorder, postorder): $O(n)$ для всех случаев.
  1. inorder: лево - корень - право
  2. preorder: корень - лево - право
  3. postorder: лево - право - корень

Упражнения

12.1-1. Начертите бинарные деревья поиска высотой 2, 3, 4, 5 и 6 для множества ключей `{1, 4, 5, 10, 16, 17, 21}`.



12.2-1. Пусть у нас имеется ряд чисел от 1 до 1000, организованных в виде бинарного дерева поиска, и мы выполняем поиск числа 363. Какая из следующих последовательностей не может быть последовательностьюпроверяемых узлов?

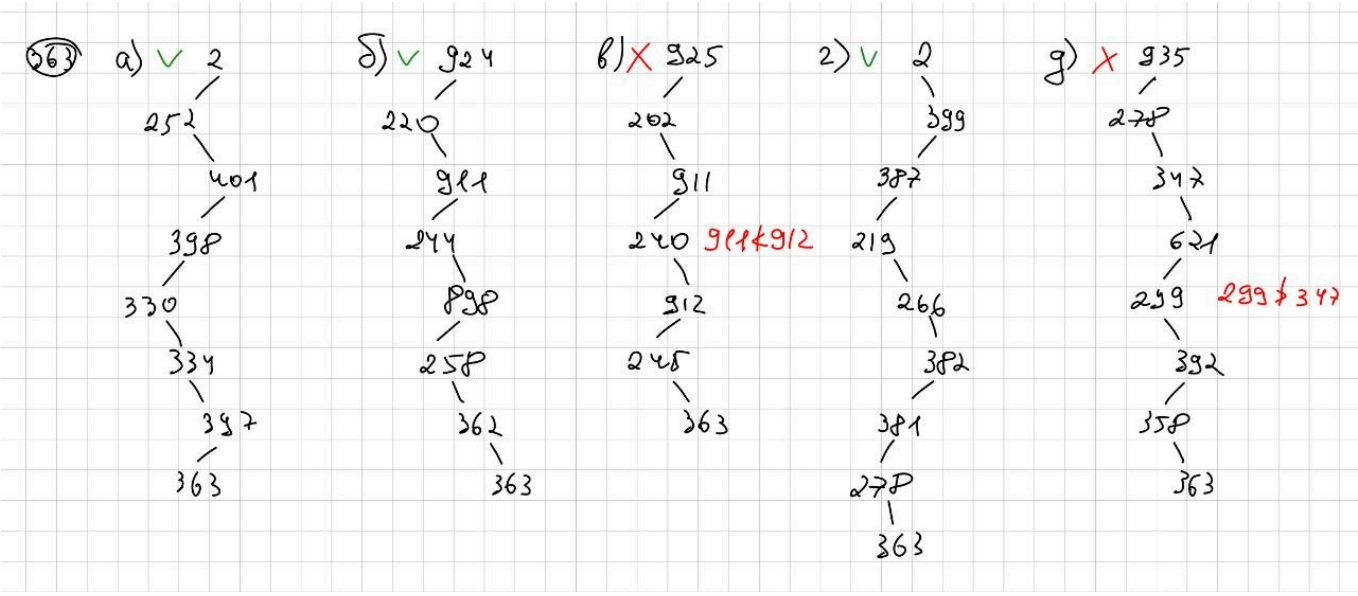
а) 2, 252, 401, 398, 330, 344, 397, 363.

б) 924, 220, 911, 244, 898, 258, 362, 363.

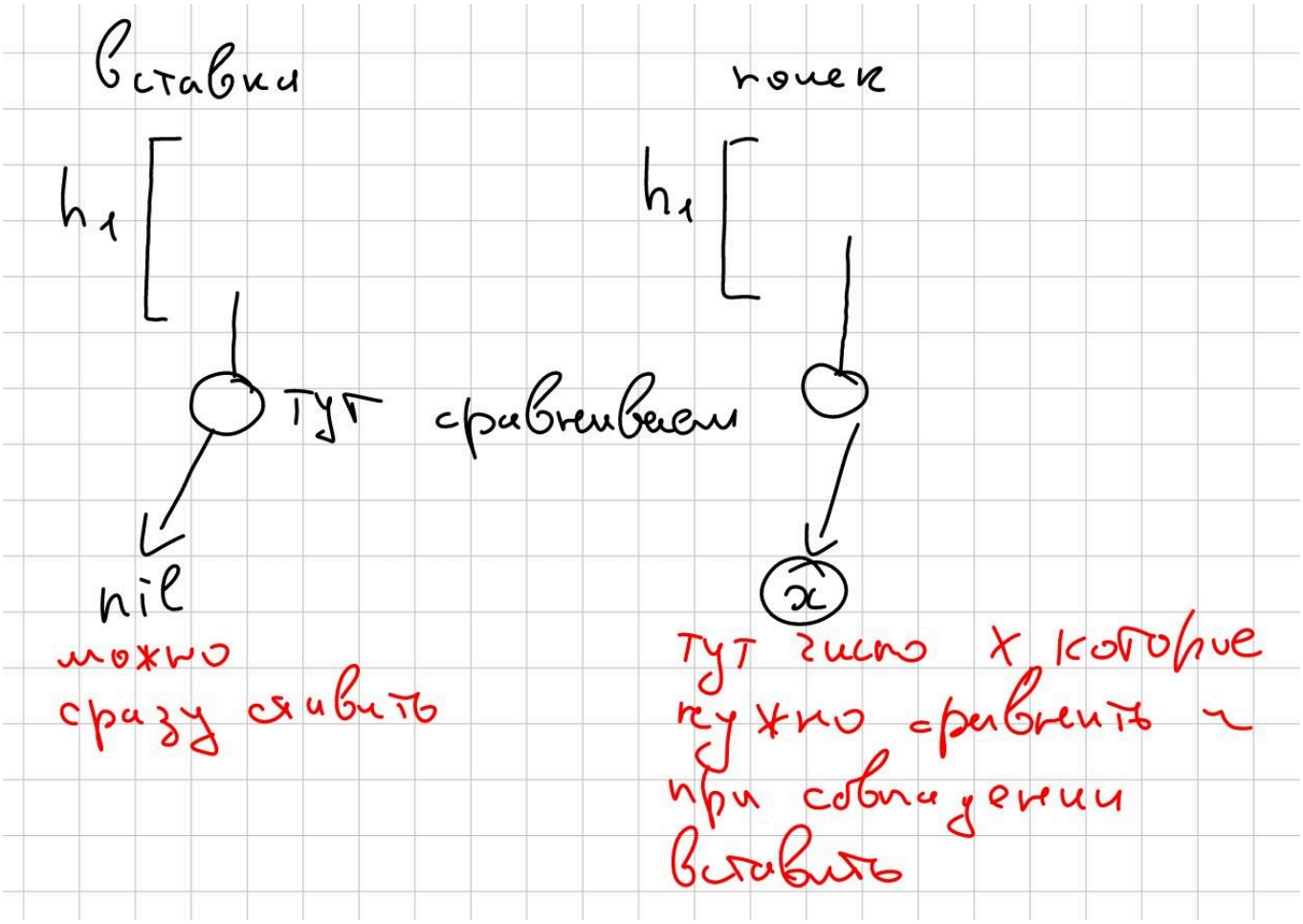
в) 925, 202, 911, 240, 912, 245, 363.

г) 2, 399, 387, 219, 266, 382, 381, 278, 363.

д) 935, 278, 347, 621, 299, 392, 358, 363.



12.3-2. Предположим, что бинарное дерево поиска строится путем многократной вставки в дерево различных значений. Покажите, что количество узлов, проверяемых при поиске некоторого значения в дереве, на один больше, чем количество узлов, проверявшихся при вставке этого значения в дерево.



12.3-3. Отсортировать множество из n чисел можно следующим образом: сначала построить бинарное дерево поиска, содержащее эти числа (вызывая процедуру `tree`

`insert` для вставки чисел в дерево одно за другим), а затем выполнить центрированный обход получившегося дерева. Чему равно время работы такого алгоритма в наилучшем и наихудшем случае?
Лучший случай: Данные поступают в таком формате, что образуется *сбалансированное* дерево. Тогда вставка n элементов будем иметь сложность $O(n \log n)$, inorder обход $O(n)$. Итого, $O(n \log n)$
Худший случай: Данные поступают уже в отсортирванном формате, тогда обраузется список, вставка n элементов в которого будет $O(n^2)$ (n элементов пройди до вставляемого места, эту операцию нужно выполнить n раз для всех элементов), inorder обход останется прежним. Итого: $O(n^2)$

12.3-5. Является ли операция удаления "коммутативной" в том смысле, что удаление `x` с последующим удалением `y` из бинарного дерева поиска приводит к тому же результирующему дереву, что и удаление `y` с последующим удалением `x`? Обоснуйте свой ответ.
Кратко: нет
Обоснование: Если у удаляемого узла ровно 2 потомка, то он будет заменён не своим прямым потомком, а максимальным левого или минимальным правого. Следовально, порядок удаления критичен: удаление одного узла может изменить структуру дерева так, что изменится приемник/родитель.

13.2-2. Покажите, что в каждом бинарном дереве поиска с n узлами имеется ровно $n-1$ возможных поворотов.
В любом дереве с n узлами имеется ровно $n-1$ ребко (у root нет ребра, прихоядщего в него). Над каждым ребром можно совершить один, и только один поворот (поднять потомка, опустить родителя). Следовательно, для $n-1$ количества рёбер можно сделать $n-1$ количество поворотов.

AVL дерево

Свойства AVL деревьев

Для любого узла дерева высота левого поддерева и высота правого поддерева отличаются не более чем на единицу.
Если разница высот по модулю равна 2 и дерево, откуда пришли, имеет высоту того же знака, то делаем один поворот, иначе два.

Операции и их сложность

- Вставка: $O(\log n)$. *Примечание:* при вставка может нарушится условие балансировки, в этом случае будет достаточно одного или двух поворотов, дальнейшая проверка окажется бессмысленной.

- Удаление: $O(\log n)$. *Примечание:* при удалении может уменьшится общая высота дерева, в таком случае нужно делать $\log n$ поворотов до самого корня.
- Поиск: $O(\log n)$

Чему равна высота дерева с n ключами? Докажите

Кратко: $h \approx O(\log n)$

Объяснение: Чтобы построить AVL-дерево высоты h с минимальным числом узлов, у его корня одно поддерево должно иметь высоту $h-1$ (чтобы обеспечить общую высоту h), а второе поддерево должно иметь высоту $h-2$ (это минимально возможная высота второго поддерева, при которой не нарушается свойство AVL, то есть разница высот равна ровно 1).

Отсюда получаем рекурсивное соотношение для количества узлов:

$$N(h) = N(h-1) + N(h-2) + 1,$$

где `+1` это сам корень.

Т.к. $N(h-1) > N(h-2) \quad => \quad N(h) > 2 \cdot N(h-2)$

т.е. $N(h) > 2^{h/2}, \quad => \quad \log N > \frac{h}{2}, \quad => \quad h < \log N$

ИЛИ из книжки:

Теорема А (Адельсон-Вельский и Ландис). *Высота сбалансированного дерева с N внутренними узлами ограничена значениями* $\lg(N+1)$ *и* $1.4405 \lg(N+2) - 0.3277$.

Доказательство. Бинарное дерево высотой h, очевидно, не может иметь более 2^h внешних узлов; таким образом, $N+1 \le 2^h$, т. е. $h \ge \lceil \lg(N+1) \rceil$ для любого бинарного дерева.

Для поиска максимального значения h рассмотрим проблему с другой стороны и зададимся вопросом о минимальном количестве узлов в сбалансированном дереве высотой h. Пусть T_h — такое дерево с наименьшим возможным количеством узлов, тогда одно из поддеревьев корня, например левое, имеет высоту $h-1$, а правое — $h-1$ или $h-2$. Поскольку дерево T_h должно иметь минимальное количество узлов, можно считать, что в силу определения T_h левое поддерево представляет собой дерево T_{h-1}, а правое — T_{h-2}. Таким образом, наименьшее возможное количество узлов среди всех сбалансированных деревьев имеет *дерево Фибоначчи* (см. определение деревьев Фибоначчи в разделе 6.2.1). Следовательно,

$$N \ge F_{h+2} - 1 > \phi^{h+2}/\sqrt{5} - 2$$

и требуемый результат получается так же, как и следствие из теоремы 4.5.3F. ∎

Красно-чёрное дерево

Свойства КЧ-деревьев:

1. Каждый узел является либо чёрным, либо красным

2. Корень дерева всегда чёрный
3. Каждый лист дерева (nil) явяется чёрным
4. Если узел красный, то оба его ребёнка чёрные
5. Для каждого узла все пути от него до листа содержит одинаковое количество чёрных узлов

Для обозначения всех листьев вместо выделение каждого раза `nil` можно создать узел `nil[T]`, цвет которого будет всегда чёрным, остальные поля любыми. Теперь вместо порождения пустых указателей можно будет использовать фиксированный узел, экономя память.

Чёрная высота $bh(x)$ - количество чёрных узлов от текущего узла не включительно до любого его листа. *Чёрная высота дерева* - чёрная высота корня.

Операции и их сложность:

- Поиск: $O(\log n)$
- Вставка: $O(h) = O(\log n)$. В дерево принято вставлять только *красные* узлы (чтобы сохрнялась чёрная высота). В случае необходимости узлы выше перекрашиваются, на это может потребоваться $O(\log n)$ перекрасок и *не более* двух поворотов.
- Удаление: $O(h) = O(\log n)$. Если удаленный узел красный, то всё супер; если чёрный, то нарушается свойство равности чёрных высот. Для второго случая требуется $O(\log n)$ перекрасок и *не более* трёх поворотов.

Гайд по вставке:

- *Случай 1: Дядя КРАСНЫЙ.*
  - Действие: Перекрашиваем отца и дядю в черный, а деда в красный. Переходим к деду и проверяем его (рекурсивно поднимаемся вверх). Повороты не нужны.
- *Случай 2: Дядя ЧЕРНЫЙ, а мы — "внутренний" внук* (например, отец — левый ребенок, а мы — правый).
  - Действие: Делаем один поворот вокруг отца, чтобы превратиться в Случай 3.
- *Случай 3: Дядя ЧЕРНЫЙ, а мы — "внешний" внук* (отец — левый, и мы — левый).
  - Действие: Делаем поворот вокруг деда, перекрашиваем отца в черный, а деда в красный. Балансировка завершена!

Чему равна высота КЧ деерва с n внутренними узлами? Почему?
Лемма: КЧ дерево с n внутренними узлами имеет высоте не более $2\log(n+1)$
Док-во: Поддерево узла x содержит минимум $2^{bh(x)} - 1$ внутренний узел. Если высота x равна 0, то узел является листом, его поддерево содержит 0 узлов. Если же x внутренний узел с положительной высотой и двумя потомками, тогда каждый потомок имеет чёрные высоты $bh(x)$ или $bh(x) - 1$. Высота узла x больше высоты его потомков, следовательно,

каждый потомок имеет минимум $2^{bh(x)-1}-1$ внутренних узлов. Значит, корень дерева имеет минимум $\left(2^{bh(x)-1}-1\right)+\left(2^{bh(x)-1}-1\right)+1=2^{bh(x)}-1$.
Зафиксируем $bh(x)$ как константу h. Тогда $n \geq 2^{h/2}-1$. Логарифмируем и получаем:

$$\log(n+1) \geq h/2, \quad \text{или} \quad h \leq 2\log(n+1)$$

Упражнения

13.1-1. Начертите в стиле рис. 13.1а полное бинарное дерево поиска высоты 3 с ключами {1, 2, …, 15}. Добавьте к нему листья NIL и раскрасьте получившееся дерево разными способами, так чтобы в результате получились красно-черные деревья с черной высотой 2, 3 и 4.

13.1-3. Определим ослабленное красно-черное дерево как бинарное дерево поиска, которое удовлетворяет красно-черным свойствам 1, 3, 4 и 5. Другими словами, корень может быть как черным, так и красным. Рассмотрите ослабленное красно-черное дерево `T`, корень которого красный. Если мы перекрасим корень дерева `T` из красного цвета в черный, будет ли получившееся в результате дерево красно-черным?
Ответ: да
Обоснование: 1) узел к или ч; 3) лист (`nil`) чёрный; 4) новых красных узлов не добавилось, чёрные узлы могут быть потомками чёрного; 5) bh дерева увеличилась на единицу, но для любого пути осталась равной.

13.1-5. Покажите, что самый длинный нисходящий путь от вершины `x` к листу красно-черного дерева имеет длину, не более чем в два раза превышающую кратчайший нисходящий путь от `x` к листу.

- Кратчайший путь от `x` до листа достигается, если подряд идут чёрные листья. Тогда $L_{min}=bh(x)$.
- Самый длинный путь будет обеспечен при постоянном чередовании чёрного и красного узлов, т.е. количество чёрных узлов будет равно количеству красных узлов и $L_{max}=2\cdot bh(x)$.
- Из двух пунктов выше получаем, что $L_{max} \leq 2 \cdot L_{min}$

13.1-6. Чему равно наибольшее возможное число внутренних узлов в красно-черном дереве с черной высотой k? А наименьшее возможное число?

- *Минимальное:* дерево только из чёрных узлов, т.е. $n_{min}=2^k-1$
- *Максимальное:* чёрные чередуются с красными, т.е. $n_{max}=2^{2k}-1$

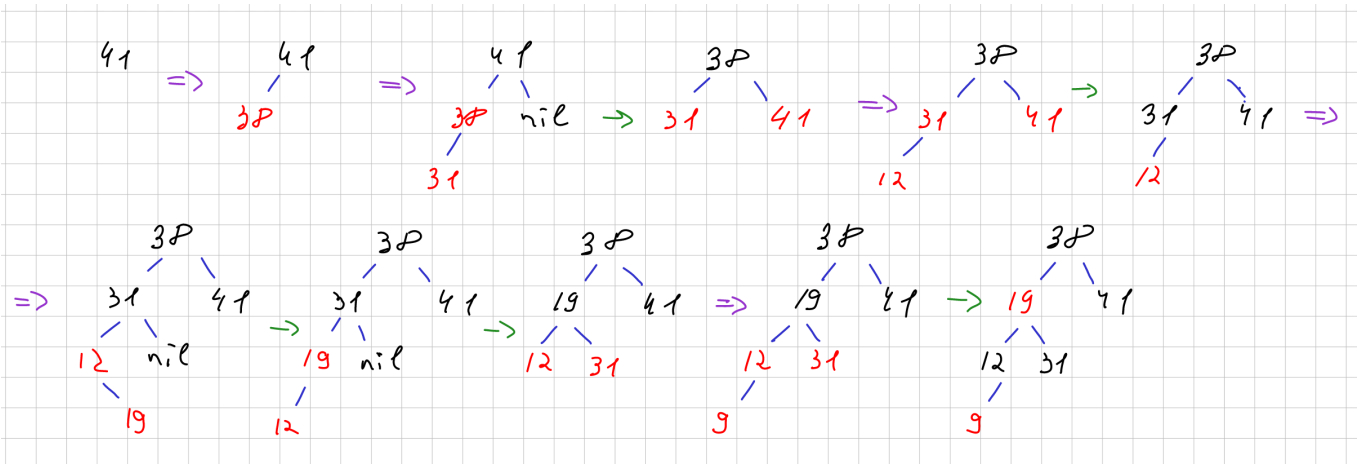
13.1-7. Опишите красно-черное дерево с n ключами с наибольшим возможным отношением количества красных внутренних узлов к количеству черных внутренних

узлов. Чему равно это отношение? Какое дерево имеет наименьшее указанное отношение, и чему равна его величина?

- *Минимальное:* очевидно, 0. Просто кч дерево без красных узлов
- *Максимальное:* Чтобы соотношение было максимальным каждый уровень должен отличаться от предыдущего и следующего цветом, при этом начиная с чёрного и заканчивая красным. Тогда на k - ом слое будет 2^k узлов. Чёрные слои - 0, 2, 4, ..., красные слои - 1, 3, 5, ... Сумма геометрической прогрессии красных узлов при стримлении к бесконечности будет в 2 раза больше аналогичной суммы чёрных. Следовательно, *отношение равно 2*.

13.3-1. В строке 16 процедуры `rb_insert` мы окрашиваем вновь вставленный узел в красный цвет. Заметим, что если бы мы окрашивали его в черный цвет, то свойство 4 не было бы нарушено. Так почему же мы не делаем этого?
Из-за нарушения свойства 5, которое гласит, что чёрная высота узла `x` должна оставаться одинаковой для любого его пути до листа. При вставка чёрного может произойти ситуация, что это условие выполняться не будет.

13.3-2. Изобразите красно-черные деревья, которые образуются при последовательной вставке ключей 41, 38, 31, 12, 19 и 8 в изначально пустое красно-черное дерево.



13.3-5. Рассмотрим красно-черное дерево, образованное вставкой n узлов при помощи процедуры `rb_insert`. Докажите, что если $n > 1$, то в дереве имеется как минимум один красный узел.
Ответ: Очевидно, что только вставленный элемент красный, и он сохраняет этот цвет даже после всех процедур восстановления свойств КЧ дерева *КРОМЕ ОДНОЙ*: `T.root.color = black`, т.е. если вставляется корень, он будет перекрашен из красного в чёрный, но тогда $n = 1$. Следовательно, $\forall n > 1$ после вставки будет *минимум* один красный узел